AIHub Concept

Document Metadata

Property	Value
Author	Enrico Goerlitz
Date	2025-12-07
Version	1.0
Status	Draft
Last Updated	2025-12-07

AIHub Vision

AIHub transforms the complexity of managing multiple AI systems into a single, unified experience. In today's AI landscape, organizations deploy agents across diverse platforms—Microsoft Foundry, n8n, LangGraph, Copilot, and custom solutions—resulting in fragmented user experiences, inconsistent monitoring, and operational silos. AIHub eliminates these challenges by providing a **Unified-UI for your AI**, where every agent, regardless of origin, converges into one cohesive platform.

The Problem We Solve

Fragmented AI Experiences
When your organization leverages multiple agent platforms, users face:

- **Inconsistent interfaces**: Each platform presents its own chat experience, creating friction and reducing productivity
- **Missing UI layers**: Custom-built agents (e.g., LangGraph multi-agent systems) often lack any user interface, requiring developers to build and maintain separate front-ends
- **Scattered monitoring**: Tracing and observability data lives in disparate systems, making it impossible to gain holistic insights into AI performance
- **Invisible autonomous operations**: Background agents and automated processes generate logs and traces that remain buried in technical infrastructure, inaccessible to business users who need operational visibility

Integration Complexity
Connecting agents from different platforms demands significant engineering effort:

- Custom API integrations for each agent system
- Bespoke authentication and authorization flows
- Redundant implementations of common features like conversation history and user management
- **Repetitive UI development**: Every new agent requires building and maintaining its own chat interface, dashboard, and monitoring tools—multiplying development effort and fragmenting the user experience

Platform Lock-In
Organizations fear being tied to a single cloud provider or agent platform, limiting flexibility as technology evolves.

Development Overhead
Engineering teams waste valuable time on undifferentiated work:

- Building chat interfaces from scratch for each agent deployment

- Implementing authentication, authorization, and user management repeatedly
- Creating custom dashboards to visualize autonomous agent activity
- Maintaining separate tracing and logging pipelines for each agent system

The AIHub Solution

One Unified Interface, Every Agent
AIHub provides a single, consistent chat experience for all your AI agents—whether they're built on enterprise platforms like Microsoft Foundry, workflow automation tools like n8n, or custom frameworks like LangGraph. Users interact with every agent through the same intuitive interface, complete with:

- **Custom widget integration** for specialized interactions (forms, data visualization, interactive components)
- **Consistent conversation history** across all agent types
- **Unified access control** via your existing identity provider (Microsoft Entra ID, Google OAuth)

Centralized Tracing and Observability
AIHub's **unified tracing framework** ingests telemetry from all your agents—conversational and autonomous—into a single database:

- **Monitor all agents** from one dashboard, regardless of their underlying platform
- **Semantic search** across traces, conversations, and agent outputs
- **Chat with your traces**: Ask natural language questions about agent behavior, performance patterns, and historical interactions
- **No lock-in**: Even agents without native tracing stores (e.g., custom LangGraph deployments) can stream telemetry to AIHub via our SDK

Cloud-Agnostic Architecture
Deploy AIHub wherever your business needs it:

- **SaaS**: Fully managed, zero-ops deployment for rapid adoption
- **Azure, AWS, or GCP**: Run AIHub in your preferred cloud environment with full control over data residency
- **On-Premises**: Deploy entirely within your data center for maximum compliance and security

Agent systems themselves remain platform-agnostic—develop and deploy them anywhere, and AIHub seamlessly integrates them through standardized APIs.

Native Support for Custom Agents
Building a specialized LangGraph multi-agent system or deploying Azure Functions-based autonomous agents? AIHub's SDK makes integration effortless:

- **Minimal code changes**: Add our SDK to send conversation data and tracing telemetry
- **No UI development required**: Instantly expose your custom agents through AIHub's chat interface
- **Built-in authentication**: Leverage AIHub's identity integration without implementing your own auth layer

Why AIHub Matters

For Business Leaders

- **Accelerate AI adoption**: Users engage with AI through a single, familiar interface—no training on multiple platforms
- **Reduce operational costs**: Consolidate monitoring, authentication, and UI development across all agents
- **Future-proof your AI investments**: Add new agent platforms or migrate existing ones without disrupting user experiences

For Engineering Teams

- **Developer velocity**: Integrate new agents in hours, not weeks, using standardized APIs and SDKs
- **Comprehensive observability**: Gain visibility into every agent invocation, from interactive chats to background processing
- **Flexible deployment**: Choose the infrastructure that aligns with your compliance, latency, and cost requirements

For End Users

- **Consistent experience**: One chat interface for every AI capability—no context switching between platforms
- **Contextual interactions**: Widgets adapt to business requirements, providing forms, visualizations, and custom UIs within conversations
- **Trustworthy AI**: Transparent tracing and audit logs ensure accountability and explainability

The AIHub Advantage

AIHub isn't just another chat interface—it's the **integration layer for your entire AI landscape**. Whether you're running enterprise agents on Microsoft platforms, automating workflows with n8n, or deploying cutting-edge custom solutions with LangGraph, AIHub provides the foundation for scalable, observable, and user-friendly AI operations.

Integrate your AI landscape. Unify your experience. Accelerate your innovation.

AIHub Overview

AIHub is a unified interface platform for deploying and managing AI chat applications powered by external agent systems. It serves as a central hub that integrates diverse agent platforms—such as Microsoft Foundry, n8n, Copilot, and others—into a single, cohesive application environment.

Key Capabilities

- **Unified Chat Interface**: Interact with multiple agent systems through a consistent chat experience, regardless of the underlying platform
- **Flexible Widget System**: Seamlessly embed custom, context-aware widgets into chat conversations to handle specialized use cases like forms, data visualization, and interactive components
- **Autonomous Agent Tracing**: Configure background agents to write tracing data directly to AIHub's centralized tracing database, providing visibility into autonomous operations alongside interactive conversations
- **Multi-Provider Identity**: Integrate with your identity provider of choice (Microsoft Entra ID, Google OAuth, and more) for secure authentication and authorization
- **Configurable Applications**: Define chat applications with specific agent configurations, share them with teams, or deploy them organization-wide with granular permission controls

Deployment Options

AIHub offers flexible deployment models to meet your infrastructure and compliance requirements:

- **SaaS**: Fully managed cloud service with minimal setup and maintenance overhead
- **Cloud Deployment**: Deploy in your own cloud environment (Azure, AWS, or GCP) for greater control over data residency and infrastructure
- **On-Premises**: Run AIHub entirely within your own data center for maximum security and compliance control

Core Value Proposition

AIHub enables you to:

1. **Build once, integrate anywhere**: Develop agents on your preferred platform and integrate them effortlessly through AIHub's standardized API
2. **Centralize agent interaction**: Manage both conversational and autonomous agents from a single interface
3. **Customize experiences**: Leverage the widget system to tailor chat interactions to specific business requirements
4. **Control access**: Use role-based permissions and group management to govern who can access, modify, and invoke applications
5. **Maintain security**: Combine identity provider authentication with application-level key management for secure data ingestion and tracing

Architecture

Technology Stack

AIHub's architecture is built on modern, scalable technologies designed to support flexible deployment across multiple cloud providers:

Database Layer

AIHub uses a **JSON-based document database** such as Azure Cosmos DB, MongoDB, or equivalent services on AWS and GCP. This approach provides:

- **Schema flexibility** through native JSON document storage
- **Low-latency access** to conversation history, application configurations, and tracing data
- **Elastic scalability** to accommodate varying workloads from small teams to enterprise-wide deployments
- **High availability** and disaster recovery capabilities

Key data models include:

- Application definitions and configurations
- Conversation threads and message history
- Autonomous agent registrations and tracing data
- User permissions and custom group memberships

Caching Layer

AIHub implements a **caching layer** to optimize performance and reduce load on the underlying database. The cache layer is critical for minimizing latency in authorization checks and frequently accessed data.

Key Use Cases:

- **Permission Resolution**: User permissions and group memberships are cached to avoid repeated lookups during authorization checks. This significantly reduces latency for API requests that require permission validation.
- **Application Configurations**: Frequently accessed application definitions are cached to accelerate conversation invocations and reduce database read operations.
- **Identity Provider Data**: User and group information retrieved from external identity providers (e.g., Microsoft Entra ID) is cached with configurable TTL to balance freshness with performance.
- **Conversation Metadata**: Recent conversation threads and their associated application references are cached for faster retrieval in active sessions.

Cache Invalidation Strategy:

- **Time-based expiration (TTL)**: Default cache entries expire after a configurable period (e.g., 5-15 minutes for permissions, 1-5 minutes for application configs)
- **Event-based invalidation**: Mutations to applications, permissions, or group memberships trigger explicit cache invalidation for affected keys
- **Manual purge**: Administrative endpoints allow selective or complete cache clearing when needed

Message Broker and Event Processing Layer

AIHub implements an **event-driven architecture** for autonomous agent tracing ingestion, decoupling trace collection from the core API.

Purpose:

External agent systems write tracing data to a message broker instead of directly calling AIHub APIs. A dedicated microservice (consumer) processes these messages asynchronously and ingests them into the database.

Supported Message Brokers:

- **Azure**: Azure Event Hubs, Azure Service Bus
- **AWS**: Amazon Kinesis, Amazon SQS
- **GCP**: Google Cloud Pub/Sub
- **Multi-Cloud/On-Premises**: Apache Kafka, RabbitMQ

Architecture Components:

1. **Message Producer (External Agent Systems)**
    - Autonomous agents send tracing data as structured messages to the configured message broker
    - Messages include agent ID, trace payload, timestamp, and correlation metadata
    - Authentication uses the agent's identity provider credentials or dedicated service principal
2. **Message Broker**
    - Acts as the event buffer between external agents and AIHub
    - Provides durability, ordering guarantees, and delivery semantics
    - Scales independently to handle high-throughput tracing workloads
3. **Tracing Consumer Microservice**
    - Subscribes to the message broker and processes tracing events
    - Validates agent identity and permissions before ingestion
    - Enriches trace data with metadata (ingestion timestamp, correlation IDs)
    - Writes validated traces to the database
    - Implements retry logic and dead-letter queue handling for failed messages

Key Benefits:

- **Asynchronous Processing**: Tracing ingestion does not block agent execution or impact AIHub API performance
- **Scalability**: Consumer microservice scales independently based on message throughput
- **Resilience**: Message broker buffers traces during outages or high-load periods
- **Decoupling**: External agents remain operational even if the consumer microservice is temporarily unavailable
- **Flexibility**: Support for multiple message broker technologies across cloud providers

Security Considerations:

- **Authentication**: Agents authenticate to the message broker using managed identity or service principal credentials
- **Authorization**: Consumer microservice validates agent permissions before writing to the database
- **Encryption**: Messages are encrypted in transit (TLS) and at rest (broker-specific encryption)
- **Audit Logging**: All trace ingestion events are logged for compliance and monitoring

Deployment Patterns:

- **SaaS**: AIHub operates a shared message broker and consumer service
- **Cloud Deployment**: Customer-managed message broker with AIHub-managed consumer microservice
- **On-Premises**: Fully customer-managed message broker and consumer infrastructure

Secrets Management Layer

Purpose:

AIHub's secrets management layer ensures secure storage and retrieval of sensitive credentials across two distinct scopes:

1. **Infrastructure Secrets**: Environment variables and configuration required for AIHub's own operation
2. **Application Secrets**: User-defined keys for external agent systems and autonomous agent authentication keys

Architecture Overview:

AIHub implements a **dual-keystore architecture** to separate concerns and maintain security boundaries:

1. AIHub Infrastructure Keystore

Manages secrets required for AIHub's core infrastructure and operation.

Stored Secrets:

- Database connection strings
- Identity provider client secrets (e.g., Microsoft Entra ID app registration credentials)
- Cache connection credentials (Redis connection strings, access keys)
- Message broker connection strings and access keys
- Internal service-to-service authentication tokens
- Encryption keys for data-at-rest protection

Implementation:

- **Azure Deployments**: Azure Key Vault
- **AWS Deployments**: AWS Secrets Manager
- **GCP Deployments**: Google Secret Manager
- **On-Premises**: HashiCorp Vault or equivalent enterprise secrets management solution

Access Pattern:

- AIHub retrieves these secrets **at startup** using managed identity
- Secrets are loaded into memory and refreshed periodically (e.g., every 24 hours)
- No API exposure—these secrets are never accessible via AIHub's public API

2. AIHub Application Keystore

Manages secrets created and used by AIHub applications and autonomous agents.

Stored Secrets:

- Application authentication keys for external agent systems (e.g., Microsoft Foundry API keys, n8n webhook tokens)
- Autonomous agent tracing keys (primary and secondary)
- Custom integration credentials configured by users

Implementation: Uses the same underlying secrets management service as the infrastructure keystore, but maintains logical separation through:

- **Separate resource/folder structure**: Application secrets stored in dedicated namespace
- **Different access policies**: Governed by AIHub's permission model rather than infrastructure access controls
- **API-driven lifecycle**: Created, updated, and rotated via AIHub API endpoints

Access Pattern:

- **Write-only via API**: Users can create/update secrets through `/api/v1/keystore/secrets/` but cannot retrieve actual values
- **Internal retrieval**: AIHub backend retrieves secret values when invoking external agents or validating autonomous agent keys
- **Metadata queries**: API returns secret metadata (name, creation date, last rotation) without exposing actual values

Security Characteristics:

- **Encryption at rest**: All secrets encrypted using provider-managed or customer-managed keys
- **Audit logging**: All access to secrets is logged for compliance and security monitoring
- **Access control**: Infrastructure secrets use cloud IAM; application secrets use AIHub's permission model
- **Key rotation**: Autonomous agent keys support dual-key rotation without service interruption
- **Principle of least privilege**: AIHub uses scoped permissions when accessing secrets (read-only for infrastructure secrets, read/write for application secrets)

Best Practices:

- **Never log secret values**: AIHub ensures secrets are never written to application logs
- **Use managed identities**: Prefer managed identity/workload identity over static credentials for accessing keystores
- **Regular rotation**: Configure automated rotation policies for long-lived secrets
- **Separate environments**: Use distinct keystores for development, staging, and production deployments
- **Monitor access patterns**: Alert on unusual secret access patterns or failed authentication attempts

Backend Layer

The backend is implemented using **FastAPI**, a modern Python web framework that provides high performance through asynchronous request handling, automatic API documentation, and type-safe request/response validation.

The backend consists of:

- **API Service**: Handles user requests, authentication, authorization, and conversation management
- **Tracing Consumer Microservice**: Processes autonomous agent tracing events from the message broker

Frontend Layer

The user interface is built with **React** and **TypeScript**, offering a type-safe, component-based architecture for building responsive chat interfaces and dynamically rendering custom widgets.

Endpoints

Applications

Applications represent AI chat configurations that enable integration with external agent systems such as Microsoft Foundry, n8n, and Copilot. Each application defines how to invoke and interact with these external services.

Application Endpoints

Method	Endpoint	Description
GET	/api/v1/applications/	List all applications
POST	/api/v1/applications/	Create a new application
GET	/api/v1/applications/permissions/	Retrieve application permissions
PUT	/api/v1/applications/permissions/	Update application permissions
DELETE	/api/v1/applications/permissions/	Delete application permissions
GET	/api/v1/applications/{id}/	Get application by ID
PATCH	/api/v1/applications/{id}/	Update application by ID
GET	/api/v1/applications/{id}/conversations/	List conversations for an application
POST	/api/v1/applications/{id}/widgets/{id}	Add widget to application by ID
DELETE	/api/v1/applications/{id}/widgets/{id}	Remove widget from application by ID

Note: Permission scope, resource type, and access level are specified in the request body or query parameters of `/permissions` endpoints.

Conversations

Conversations represent active chat sessions between users and configured applications. Each conversation maintains message history and supports real-time invocation of the underlying agent system.

Conversation Endpoints

Method	Endpoint	Description
GET	/api/v1/conversations/permissions/	Retrieve conversation permissions
PUT	/api/v1/conversations/permissions/	Update conversation permissions
DELETE	/api/v1/conversations/permissions/	Delete conversation permissions
POST	/api/v1/conversations/{id}/invoke/	Invoke the agent with a new message
GET	/api/v1/conversations/{id}/messages/	Retrieve conversation message history
PUT	/api/v1/conversations/{id}/messages/{id}	Update a specific message

Note: Permission scope, resource type, and access level are specified in the request body or query parameters of `/permissions` endpoints.

Note: Regular users may only modify their own user messages. System-level accounts with elevated permissions can modify assistant messages. Configure access via custom groups as needed.

Widgets

Widgets enable the integration of custom UI components into chat conversations. Widgets can be either default components provided by AIHub or custom widgets embedded via iframe. Applications can be configured to include specific widget types along with instructions that guide the agent on how and when to integrate these widgets into the conversation.

Method	Endpoint	Description
GET	/api/v1/widgets/	List all available widgets
POST	/api/v1/widgets/	Register a new custom widget
GET	/api/v1/widgets/{id}	Get widget by ID
PATCH	/api/v1/widgets/{id}	Update widget configuration
DELETE	/api/v1/widgets/{id}	Delete a custom widget

Note: This section is currently **work in progress**. Detailed specifications for default widget types, custom widget integration patterns, and application-widget assignment mechanisms will be added in future updates.

Keystores

Keystores securely manage secrets and API keys required by applications to communicate with external agent systems. The keystore API provides metadata operations without exposing actual secret values.

Keystore Endpoints

Method	Endpoint	Description
GET	/api/v1/keystore/secrets/	List secret metadata (no actual keys returned)
PUT	/api/v1/keystore/secrets/	Create or update a secret
GET	/api/v1/keystore/secrets/{id}	Get specific secret metadata (no actual keys returned)

Note: Permission scope, resource type, and access level are specified in the request body or query parameters of `/permissions` endpoints.

Autonomous Agents and Tracing

Autonomous Agents operate independently in the background and write tracing data to AIHub's centralized tracing database. Each agent is issued two rotatable keys (primary and secondary) for secure data ingestion.

Autonomous Agent Endpoints

Method	Endpoint	Description
GET	/api/v1/autonom/agents/	List all autonomous agents
POST	/api/v1/autonom/agents/	Register a new autonomous agent
GET	/api/v1/autonom/agents/permissions/	Retrieve autonom agents permissions

Method	Endpoint	Description
PUT	/api/v1/autonom/agents/permissions/	Update autonom agents permissions
DELETE	/api/v1/autonom/agents/permissions/	Delete autonom agents permissions
GET	/api/v1/autonom/agents/{id}/tracings/	Retrieve tracing entries for an agent
POST	/api/v1/autonom/agents/{id}/tracings/	Create a new tracing entry
GET	/api/v1/autonom/agents/{id}/tracings/{id}	Get a specific tracing entry
POST	/api/v1/autonom/agents/{id}/tracings/{id}	Append data to an existing tracing entry

Note: Permission scope, resource type, and access level are specified in the request body or query parameters of `/permissions` endpoints.

Identity

Identity endpoints provide access to user and group information from both the configured identity provider and AIHub's custom group management system.

Identity Provider Endpoints

Method	Endpoint	Description
GET	/api/v1/identity/provider/users	List users from identity provider
GET	/api/v1/identity/provider/groups	List groups from identity provider

Custom Groups Endpoints

Method	Endpoint	Description
GET	/api/v1/identity/custom/groups/	List all custom groups
POST	/api/v1/identity/custom/groups/	Create a new custom group
GET	/api/v1/identity/custom/groups/permissions/	Retrieve custom groups permissions
PUT	/api/v1/identity/custom/groups/permissions/	Update custom groups permissions
DELETE	/api/v1/identity/custom/groups/permissions/	Delete custom groups permissions
GET	/api/v1/identity/custom/groups/{id}	Get custom group by ID
PATCH	/api/v1/identity/custom/groups/{id}	Update custom group by ID
DELETE	/api/v1/identity/custom/groups/{id}	Delete custom group by ID
GET	/api/v1/identity/custom/groups/{id}/users	List users in a custom group
PUT	/api/v1/identity/custom/groups/{id}/users	Add users to a custom group
DELETE	/api/v1/identity/custom/groups/{id}/users/{id}	Remove a user from a custom group

Authentication Concept

AIHub implements a **unified authentication system** based entirely on identity provider integration, ensuring consistent security policies across all user and service interactions.

User Authentication (Interactive Sessions)

Identity Provider Integration

AIHub integrates with enterprise identity providers to authenticate end users:

- **Supported Providers**: Microsoft Entra ID, Google OAuth, and other OAuth 2.0/OIDC-compliant providers
- **Authentication Flow**: Users authenticate via the configured identity provider and receive a JWT bearer token
- **Token Validation**: All API requests include the bearer token in the `Authorization` header, which AIHub validates against the identity provider

Example Request:

```
Authorization: Bearer <jwt-token>
```

Permission Resolution

After successful authentication, AIHub resolves user permissions by:

1. **Retrieving user identity** from the validated JWT token (user ID, email, groups)
2. **Querying group memberships** from both:
    - Identity provider groups (e.g., Microsoft Entra ID groups)
    - AIHub custom groups (managed within the application)
3. **Resolving permissions** assigned to those groups
4. **Caching resolved permissions** to optimize subsequent authorization checks

Service Authentication (Autonomous Agents)

Identity Provider Authentication

Autonomous agents use the **same identity provider authentication** as interactive users:

- Agents must authenticate as a valid **service principal or managed identity** via the identity provider
- Bearer token is required in the `Authorization` header:

  ```
  Authorization: Bearer <jwt-token>
  ```

- This ensures the calling entity is authenticated within the organization's identity system

Permission-Based Authorization

Authorization for autonomous agents follows the same permission model as users:

1. **Service principal identity** is extracted from the validated JWT token
2. **Group memberships** are resolved (identity provider groups and/or AIHub custom groups)
3. **Permissions** assigned to those groups determine what operations the agent can perform
4. **Resource-specific permissions** control which autonomous agent resources can be accessed

Example: To allow a service principal to write tracing data to a specific autonomous agent:

- Assign the service principal to a group

- Grant that group the permission: `autonom-agents/{id}:update`

Best Practices for Service Principals

- **Use managed identities** when deploying autonomous agents in Azure, AWS, or GCP
- **Create dedicated service principals** for each autonomous agent or agent system
- **Apply principle of least privilege**: Grant only the minimum required permissions
- **Rotate service principal credentials regularly** according to your organization's security policies
- **Monitor service principal activity** via audit logs and alert on anomalous behavior
- **Use separate service principals** for development, staging, and production environments

Keystore Authentication

Application secrets stored in the keystore are protected by:

- **User authentication** via identity provider
- **Permission-based access control** requiring appropriate keystore permissions
- **Write-only secret storage**: Secrets can be created/updated but never retrieved via API (only metadata is returned)
- Secrets are encrypted at rest in the underlying database

Session Management

- **Stateless authentication**: All requests are validated independently using JWT tokens
- **Token expiration**: Managed by the identity provider; expired tokens are rejected
- **Permission cache TTL**: User permissions are cached with configurable expiration (typically 5-15 minutes) and invalidated on permission changes
- **No server-side sessions**: AIHub does not maintain server-side session state, ensuring horizontal scalability

Security Best Practices

1. **Always use HTTPS** in production deployments to protect tokens in transit
2. **Configure short token lifetimes** in your identity provider (e.g., 1-4 hours)
3. **Implement refresh token rotation** for long-lived client sessions
4. **Rotate autonomous agent keys regularly** (e.g., every 90 days)
5. **Monitor failed authentication attempts** via Azure Monitor or equivalent logging
6. **Use service principals** for autonomous agents rather than user accounts
7. **Apply principle of least privilege** when assigning permissions to groups and agents

Permission Model

AIHub uses a flexible permission system that supports granular access control across all resources. Permissions can be assigned to identity provider groups, custom groups, or predefined role groups.

Permission Structure

Permissions follow the pattern `<resource>:<action>` or `<resource>/{id}:<action>` for resource-specific access.

All Resources

- `*:*` — Full access to all resources and actions

Applications

- `applications:create` — Create new applications
- `applications/*:*` — Full access to all applications
- `applications/{id}:read` — Read a specific application
- `applications/{id}:update` — Update a specific application
- `applications/{id}:delete` — Delete a specific application

Conversations

- `conversations:create` — Create new conversations
- `conversations/*:*` — Full access to all conversations
- `conversations/{id}:read` — Read a specific conversation
- `conversations/{id}:update` — Update a specific conversation
- `conversations/{id}:delete` — Delete a specific conversation
- `conversations/{id}:invoke` — Invoke the agent in a specific conversation

Autonomous Agents

- `autonom-agents:create` — Register new autonomous agents
- `autonom-agents/*:*` — Full access to all autonomous agents
- `autonom-agents/{id}:read` — Read a specific autonomous agent
- `autonom-agents/{id}:update` — Update a specific autonomous agent
- `autonom-agents/{id}:delete` — Delete a specific autonomous agent

Identity Custom Groups

- `custom-groups:create` — Create new custom groups
- `custom-groups/*:*` — Full access to all custom groups
- `custom-groups/{id}:read` — Read a specific custom group
- `custom-groups/{id}:update` — Update a specific custom group
- `custom-groups/{id}:delete` — Delete a specific custom group

Permission Groups

AIHub supports multiple approaches to organizing permissions:

Predefined Role Groups

- **VIEWER** — Read-only access across all resources
- **CONTRIBUTOR** — Read and write access to applications and conversations
- **ADMIN** — Full administrative access (`*:*`)

Resource-Specific Admin Groups

- **APPLICATION_ADMIN** — Full access to applications
- **CONVERSATIONS_ADMIN** — Full access to conversations
- **IDENTITY_CUSTOM_GROUPS_ADMIN** — Full access to custom group management

Identity Provider Groups

Use groups from your configured identity provider (e.g., Microsoft Entra ID) to assign permissions based on existing organizational structures.

AIHub Custom Groups

Create and manage custom groups within AIHub for fine-grained permission assignment independent of your identity provider.

Widget Integration

The AI Chat Widget System enables the integration of custom frontend components into chat applications through a canvas-based approach. These widgets can be developed per customer to accommodate specific requirements such as specialized forms.

LLM Response Format

The Language Model (LLM) embeds widgets into its response using a specific delimiter pattern:

Delimiter: `$%_WIDGET_%$`

Widgets are enclosed between two instances of this delimiter. The LLM inserts widget definitions directly into the conversational response text.

Example LLM Response:

```
Please fill out this form so we can proceed:

$%_WIDGET_%$
{
    "type": "WIDGET_TYPE",
    "structure": {
        ...
    }
}
$%_WIDGET_%$

Thank you.
```

Widget Structure

All widget definitions follow a consistent JSON structure with two main properties:

- `type`: Identifies the widget component to render (e.g., form type, chart type)
- `structure`: Contains the widget-specific configuration and data

Message Response Format

When a message contains widgets, the response follows this JSON structure:

```
{
    "content": "The full message text including widget delimiters",
    "widgets": [
        {
            "type": "WIDGET_TYPE",
            "structure": {
                ...
            },
            "position": 45
        }
```

```
    ]
}
```

Properties:

- `content`: The complete message text as returned by the LLM, including widget delimiters
- `widgets`: Array of parsed widget objects with:
  - `type`: Widget type identifier
  - `structure`: Widget configuration object
  - `position`: Character indices indicating where the widget appears in the content string

Weitere überlegungen

- semantic search enablen in applications, in autonomous agents, etc
- Tenant support
- als SASS Lösung bereitstellen

Warum ChatHistorie und Tracing in den AIHub integrieren

Eine zentrale konzeptionelle Frage ist, ob Chat-Historie und Tracings direkt in die AIHub-Datenbank integriert werden sollen oder ob stattdessen immer der Store der jeweiligen Agent-Plattform genutzt und Daten on-the-fly in die benötigte Struktur übersetzt werden sollen.

Pro-Argumente für die Integration in AIHub

1. **Heterogene Store-Qualität**: Nicht jedes Agent-System bietet einen ausgereiften Conversation Store mit vergleichbaren Funktionen und Leistungsmerkmalen.
2. **Fehlende Suchfunktionen**: Viele Agent-Plattformen (z.B. n8n) bieten keine semantische Suche, erweiterte Filteroptionen oder andere moderne Such- und Analysefunktionen.
3. **Plattformübergreifende Flexibilität**: Durch zentrale Datenhaltung können neue Features unabhängig von den Fähigkeiten einzelner Agent-Plattformen entwickelt und plattformübergreifend bereitgestellt werden.
4. **Erweiterte Suchfunktionen**: Semantic Search kann für Applications, Conversations und Tracings einheitlich aktiviert werden, ohne auf externe Stores angewiesen zu sein.
5. **Innovative Funktionen**: Features wie "Chat with your Traces" – semantische Analyse und Konversation über historische Tracing-Daten – sind nur mit zentraler Datenhaltung effizient umsetzbar.
6. **Support für Custom Agents**: Selbst entwickelte Agents (z.B. mit LangGraph) haben oft gar keinen eigenen Tracing- oder Logging-Store und können nahtlos integriert werden.
7. **Einheitliche Datenstruktur**: Eine vordefinierte JSON-Struktur ermöglicht konsistentes Querying, Auswertung und Visualisierung über alle Agent-Systeme hinweg.
8. **Integration autonomer Logs**: Logs aus autonomen Agents (z.B. Azure Functions) lassen sich ohne zusätzliche externe Logging-Infrastruktur direkt in AIHub aufbereiten und anzeigen.
9. **Zukunftssicherheit**: AIHub kann über ein reines Chat-UI hinauswachsen – z.B. durch Dev-Komponenten, direktes Entwickeln und Deployen von Custom Widgets innerhalb der Plattform.
10. **Unified-UI Vision**: Ein zentraler Datenstore ermöglicht ein konsistentes, einheitliches UI-Erlebnis und erlaubt dem Entwicklerteam, sich auf Backend-Innovation zu konzentrieren.

Contra-Argumente für die Integration in AIHub

1. **Doppelte Datenhaltung**: Daten werden sowohl im Agent-Store als auch im AIHub gespeichert, was zu Redundanz und potenziellem Synchronisationsbedarf führt.
2. **Erhöhte Infrastrukturkosten**: Zusätzliche Speicher- und Datenbankressourcen in AIHub erhöhen die Betriebskosten.
3. **Wartungsaufwand**: Zusätzliche Microservices (z.B. Tracing Consumer) und Datenbankschemata erfordern kontinuierliche Wartung und Updates.

Entscheidungsmatrix

Kategorie	Kriterium	Integration in AIHub	On-the-fly Übersetzung	Gewichtung	Bewertung (1-5)
Funktionalität	Erweiterte Suchfunktionen (Semantic Search)	✅ Nativ möglich	⚠ Abhängig von externen Stores	Hoch	AIHub: 5, Extern: 2
Funktionalität	Support für Custom Agents ohne Store	✅ Vollständig unterstützt	❌ Nicht möglich	Hoch	AIHub: 5, Extern: 1
Funktionalität	Einheitliche Datenstruktur	✅ Garantiert	⚠ Übersetzungsaufwand	Mittel	AIHub: 5, Extern: 3
Funktionalität	Innovative Features (Chat with Traces)	✅ Einfach umsetzbar	❌ Technisch komplex	Hoch	AIHub: 5, Extern: 2
Skalierbarkeit	Unterstützung für autonome Agents	✅ Zentralisiert	⚠ Verteilte Datenquellen	Hoch	AIHub: 5, Extern: 3
Kosten	Infrastrukturkosten	⚠ Höhere Speicherkosten	✅ Niedrigere Kosten	Mittel	AIHub: 3, Extern: 5
Kosten	Entwicklungs- und Wartungsaufwand	⚠ Zusätzliche Services	✅ Geringer	Mittel	AIHub: 3, Extern: 4
Komplexität	Datensynchronisation	⚠ Erforderlich	✅ Nicht erforderlich	Mittel	AIHub: 3, Extern: 5
Flexibilität	Plattformübergreifende Features	✅ Unabhängig	❌ Limitiert	Hoch	AIHub: 5, Extern: 2
Flexibilität	Zukunftssicherheit (neue Features)	✅ Volle Kontrolle	⚠ Eingeschränkt	Hoch	AIHub: 5, Extern: 3

Gewichtete Gesamtbewertung

Integration in AIHub: **4.5/5** (gewichtet nach Priorität für Funktionalität und Flexibilität)
On-the-fly Übersetzung: **2.9/5** (gewichtet nach Priorität für Funktionalität und Flexibilität)

Empfehlung

Die Integration von Chat-Historie und Tracings direkt in AIHub wird **stark empfohlen**, da:

1. Die Vision eines **Unified-UI für AI** zentrale Datenhaltung erfordert
2. Innovative Features wie **Semantic Search** und **Chat with Traces** nur so effektiv umsetzbar sind

3. Die **Flexibilität** für zukünftige Features und Custom-Agent-Integration höher gewichtet wird als Infrastrukturkosten
4. Die **Heterogenität externer Agent-Stores** eine konsistente UX nur über zentrale Datenhaltung ermöglicht

Die erhöhten Infrastrukturkosten und der Wartungsaufwand sind durch die strategischen Vorteile gerechtfertigt, insbesondere wenn AIHub als langfristige Plattform für AI-Integration positioniert wird.

TODOs

KONZEPT: Unified-UI for your AI AI-Integration Platformü

Marketing-Sprech: Integrate you AI-Landscape into our ONE UNIFIED-AI AI-Hub Platform. Use the experience of our „Unified-UI for your AI" to integrate various of AI-Agents such as … . Or trace your ai agent outputs to our unified tracing framework an monitor different AI-Agent from different platforms in ONE UNIFIED-UI. Chat with your traces, gain insights and improve blabla

Welches Problem wird gelöst?

- Wenn ich verschiedene Agentsysteme verwende, habe ich verschiedene Chat-experiences und je nach Technologie ggf. gar kein UI
- Integration verschiedener Agentsysteme in ein UI
- Plattformunabhänig. Sowohl die Agentsysteme können überall entwickelt werden, als auch der AIHub selbst kann auf den verschiedenen Cloud-Platzformen (Azure, AWS, GCP) deploed werden

Was ist mit agents, die keine. Store haben? Man will nen Langgraph Multi-Agent bauen und dediziert deployen. Dieser hat keinen Store oder müsste extra implementiert werden: hier kann man dann einfach über unser aihub-SDK simpel auch diesen Agent einbinden (Thema: Custom Lösungen